
GIT Y FLUJO DE TRABAJO CON AGENTES

Introducción a la ciencia de datos

Fernanda Sobrino

2026

¿Por qué git?

El problema de siempre

- ▶ Tú y tu compañera trabajan sobre el mismo análisis
- ▶ Ella cambia `limpieza.py`, tú cambias `limpieza.py`, alguien sube el archivo al final
- ▶ Resultado: uno de los dos **borró el trabajo del otro** y nadie se dio cuenta
- ▶ La versión casera del control de versiones:

`analisis.py`

`analisis_v2.py`

`analisis_v2_final.py`

`analisis_v2_final_ESTESI.py`

El problema nuevo

- ▶ Le pides a un agente “limpia los datos y ajusta el modelo”
- ▶ En 40 segundos toca **9 archivos**, borra dos columnas y cambia un merge
- ▶ Corre sin errores. Los resultados cambiaron. ¿Por qué?
- ▶ Sin git no tienes forma de contestar esa pregunta ni de regresar

Git importa más con agentes

- ▶ Antes: git servía para colaborar y no perder trabajo
- ▶ Ahora además es tu **herramienta de auditoría**: el diff es la lista exacta de lo que el agente cambió
- ▶ Un commit chico antes de pedirle algo al agente es un punto de retorno gratuito
- ▶ Regla: **nunca le pidas nada a un agente sobre un repositorio sucio**

Git básico

¿Qué es git?

- ▶ Software de control de versiones: guarda **fotos (snapshots)** de tu proyecto a lo largo del tiempo
- ▶ Es distribuido: tu copia local contiene la historia completa, no solo la última versión
- ▶ Puedes ver qué cambió, quién lo cambió, cuándo, y regresar a cualquier punto
- ▶ GitHub es un *servicio* que hospeda repositorios de git (el del curso vive ahí); git funciona sin él

Vocabulario mínimo

- ▶ **repositorio**: la carpeta del proyecto que git está vigilando
- ▶ **commit**: una foto del repositorio, con mensaje y autor
- ▶ **branch (rama)**: una línea de trabajo paralela; `main` es la principal
- ▶ **remote**: la copia en la nube (`origin`)
- ▶ **push / pull**: subir tus commits / bajar los de los demás
- ▶ **clone**: traer una copia completa del repositorio a tu máquina

El ciclo básico

```
git status          # ¿en qué estado estoy?  
git diff            # ¿qué cambió y no he guardado?  
git add limpieza.py # preparo lo que voy a guardar  
git commit -m "arreglo el merge de municipios"  
git push origin mi-rama # lo mando a la nube
```

- ▶ git status antes y después de todo — es gratis y te salva
- ▶ Si no sabes en qué estado estás, no hagas nada más

Ramas: por qué

- ▶ Una rama es una copia de main donde puedes experimentar sin romper nada
- ▶ main siempre debe estar en un estado que funcione
- ▶ Las ramas sustituyen a `analisis_v2_final_ESTE_SI.py`

```
git switch -c limpieza-encuesta # crea la rama y me mueve  
# ... trabajo, commits ...  
git push -u origin limpieza-encuesta
```

- ▶ Al terminar: *pull request* / *merge request*, alguien lo revisa, se integra a main

Deshacer

- ▶ **Cambios sin commit:** `git restore archivo.py` los tira a la basura
- ▶ **Un commit ya hecho:** `git revert <hash>` crea un commit nuevo que **invierte** los cambios
 - ▷ No borra historia: puedes auditar qué pasó y por qué se deshizo
- ▶ Volver a ver cómo estaba algo: `git log` y `git show <hash>`
- ▶ Lo que nunca hiciste commit, git no lo puede recuperar

El flujo con agentes

El diff es la unidad de revisión

- ▶ El agente no te entrega “un cambio”: te entrega un conjunto de cambios
- ▶ La única forma honesta de aceptarlo es **leer el diff completo**

```
git diff                # cambios que aún no preparo
git diff --stat         # resumen: archivos y líneas
git diff main..mi-rama  # todo lo que agrega mi rama
```

- ▶ Si el diff es demasiado grande para leerlo, el problema es el tamaño de la petición, no el diff

El flujo, en cinco pasos

1. **Rama limpia:** `git switch -c tarea-8-clustering`, `git status` sin pendientes
2. **Especifica:** una tarea acotada, con datos, criterio y qué esperas ver
3. **El agente trabaja**
4. **Revisas el diff:** archivo por archivo; lo que no entiendas, lo preguntas *antes* de aceptarlo
5. **Commit chico** con mensaje claro de *qué* y *por qué*

Repites. Un ciclo, un commit.

Commits chicos: por qué

- ▶ Un commit por idea: “escalo después del split”, no “cambios varios”
- ▶ Si algo se rompe, `git log` te dice **cuál** de los diez pasos lo rompió
- ▶ Un commit gigante generado por un agente es imposible de revisar y de revertir parcialmente
- ▶ Mensaje útil: arreglo fuga de datos: `StandardScaler` ahora se ajusta solo en `train`
- ▶ Mensaje inútil: `update`, `fix`, cambios del agente

Qué nunca hacer

- ▶ **Comitear código que no entiendes.** Si no lo puedes explicar, no lo firmas
- ▶ `git push --force` sobre una rama compartida: borra el trabajo de los demás
- ▶ Comitear datos pesados, llaves, contraseñas o `.env` (usa `.gitignore`)
- ▶ Trabajar directo en `main` mientras un agente edita archivos
- ▶ Aceptar un diff “porque las pruebas pasaron”: las pruebas también las escribió el agente

Cierre del curso

**Especificar → el agente ejecuta →
verificar el diff → firmas tú**

- ▶ El commit lleva **tu** nombre, no el del agente: en términos de responsabilidad, tú escribiste ese código
- ▶ “Lo generó la IA” no es defensa — ni aquí, ni en tu trabajo
- ▶ El agente pone velocidad; tú pones el criterio
- ▶ Las cuatro etapas requieren saber ciencia de datos: para especificar bien, y sobre todo para verificar

Cómo trabajar con un agente sin perder el control

1. **Empieza limpio:** `git status` sin cambios pendientes antes de pedir nada
2. **Rama por tarea**, nunca directo en `main`
3. **Commits chicos**, uno por idea, con mensaje que diga qué y por qué
4. **Lee el diff completo** antes de cada commit — `git diff --stat` primero, luego archivo por archivo
5. **No comitees lo que no entiendes:** pídele al agente que te lo explique, o bórralo
6. **Nunca** `--force` sobre trabajo compartido; para deshacer usa `git revert`